

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

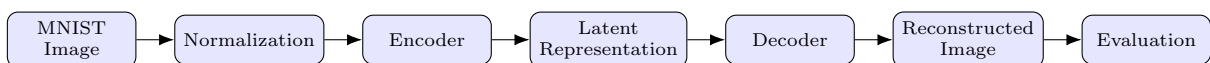
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

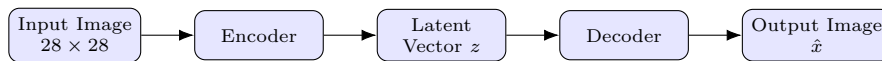
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

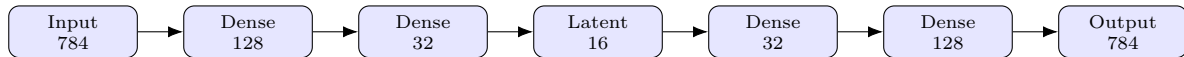
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



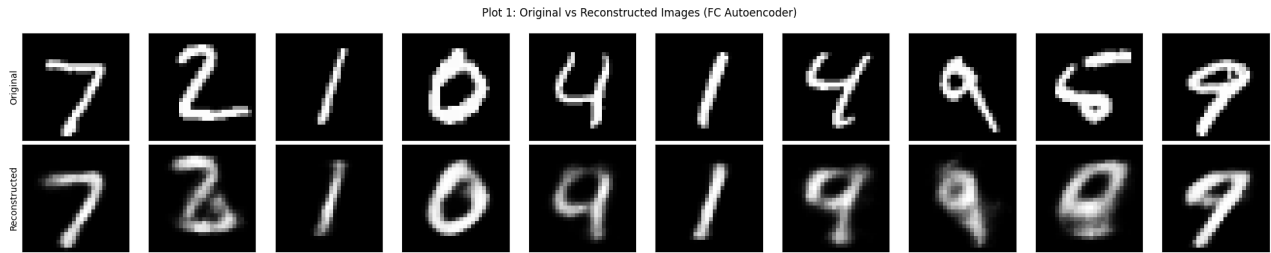
Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

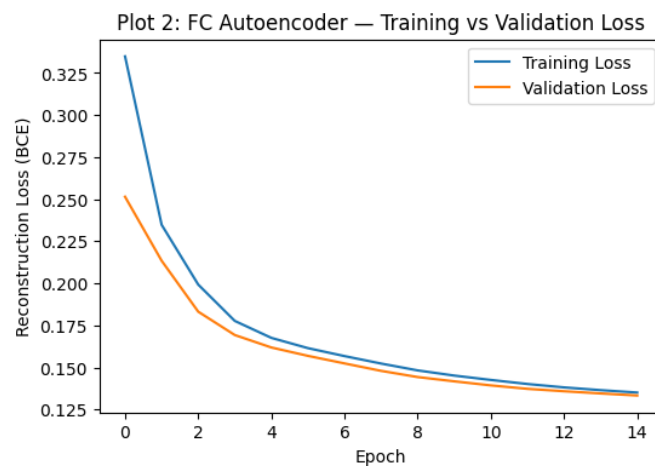
Plot 1: Original versus Reconstructed Images



Inference: The FC autoencoder keeps the overall digit shape and most digits are easy to recognise. Fine details like thin strokes and small loops are lost and the outputs look blurry, especially around stroke edges. Simple digits such as 1 are reconstructed better than digits with curves and loops like 8, 5 and 3, which come out smudged.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss



Inference: Both training and validation loss fall from about 0.33 to 0.135 over 15 epochs and stay very close to each other. There is no overfitting since the validation loss is never above the training loss. The curves are flattening but still slowly going down, so the model is close to converging but could improve with more epochs.

9. Reconstruction Metrics

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

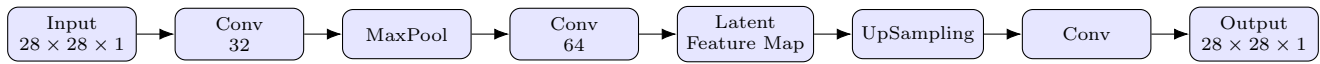
<https://scikit-image.org/docs/stable/api/skimage.metrics.html>

Metric	Value
Test MSE	0.0258
Test MAE	0.0659
Mean SSIM	0.6979

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```

Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
  
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

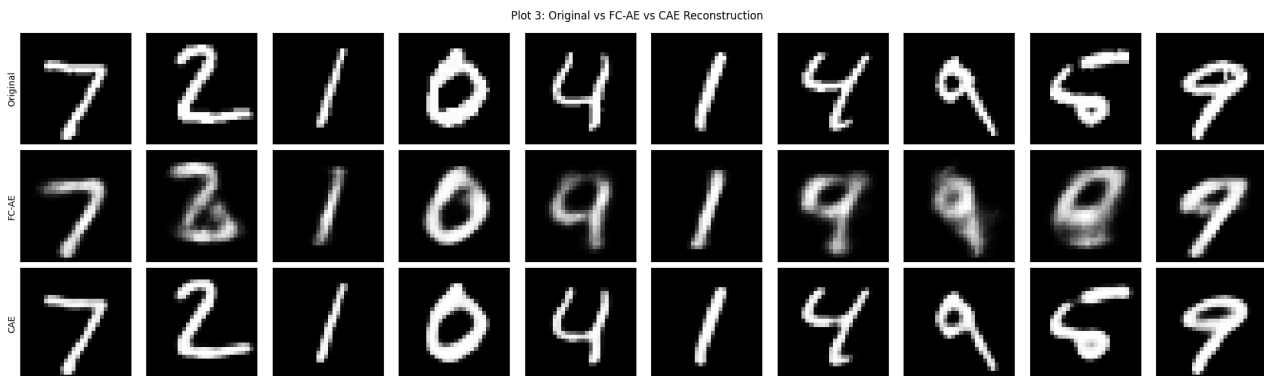
Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.0258	0.0659	0.6979	211,040
Convolutional AE	0.0029	0.0158	0.9717	74,497

Plot 3: Reconstruction Comparison

Display:

Original → FC-AE Reconstruction → CAE Reconstruction.



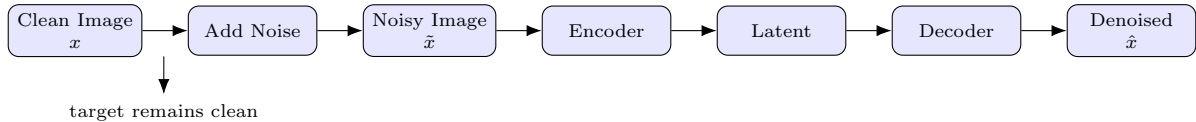
Inference: The CAE reconstructions are much sharper and almost identical to the originals, while the FC-AE outputs are blurry. The CAE gets about 9 times lower MSE (0.0029 vs 0.0258) and SSIM of 0.97 vs 0.70, even with fewer parameters (74,497 vs 211,040). This is because convolutions look at local neighbourhoods and share weights, so strokes and edges are preserved, whereas flattening the image throws away the 2-D structure.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image. The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

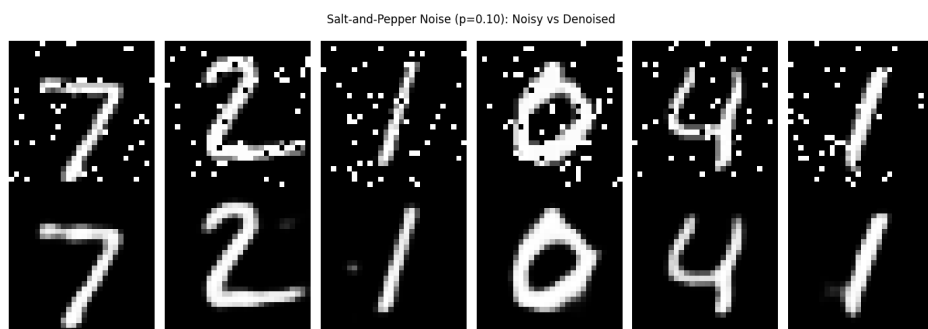
Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$



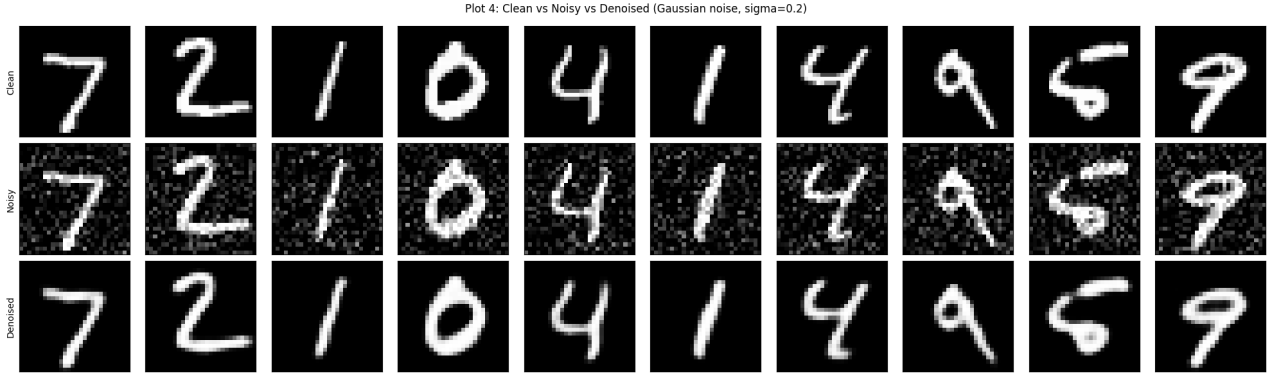
For $p = 0.10$, the denoiser gives $\text{MSE} = 0.0059$, $\text{MAE} = 0.0237$ and $\text{SSIM} = 0.9262$ on the test set. The isolated black/white pixels are removed and the digits are recovered clearly.

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.



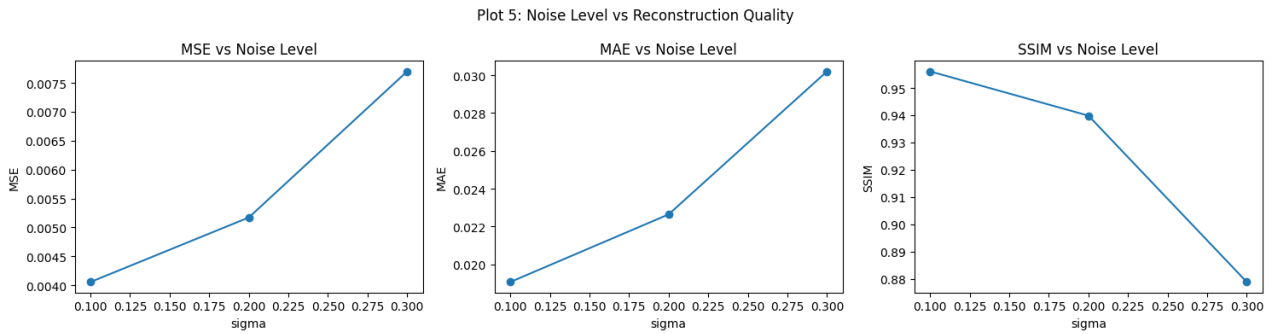
Inference: The denoised images are much cleaner than the noisy inputs (Gaussian noise, $\sigma = 0.2$) and the digit shapes are recovered well. Some fine stroke details are slightly smoothed. This shows the model learned to remove noise and keep the digit structure.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

Noise Level	MSE	MAE	SSIM
0.1	0.0041	0.0191	0.9560
0.2	0.0052	0.0226	0.9398
0.3	0.0077	0.0302	0.8789



Inference: As the noise level increases from 0.1 to 0.3, MSE goes from 0.0041 to 0.0077 and MAE from 0.0191 to 0.0302, while SSIM drops from 0.956 to 0.879. The error rises faster at higher noise because more of the stroke information is hidden. Even at $\sigma = 0.3$ the model still recovers the major digit structure, so it denoises well but loses some fine detail as corruption increases.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

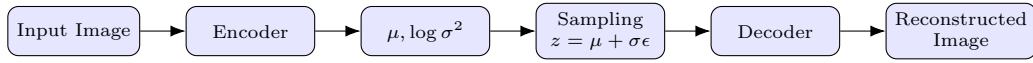
$$q_{\phi}(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

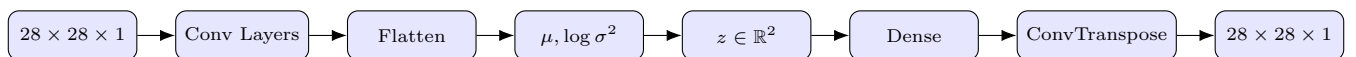
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



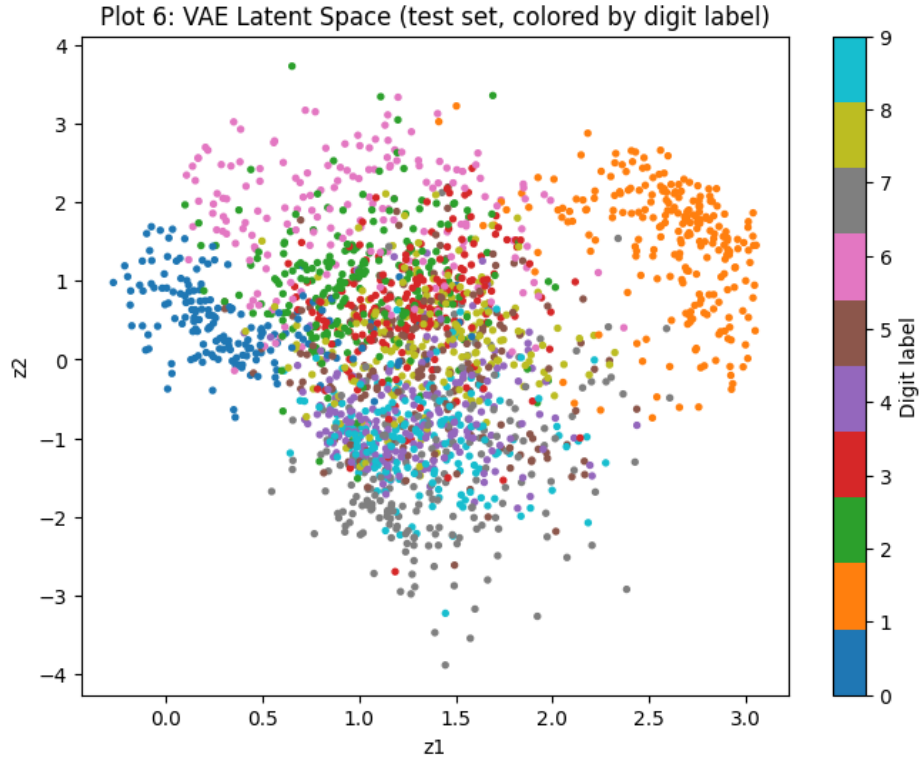
18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.



Inference: Some digits form fairly separate regions: 0 is on the left, 1 is on the right and 6 is at the top. Digits like 4, 7 and 9 overlap at the bottom, and 2, 3, 5 and 8 overlap in the centre. This is because the VAE is trained without labels and only visually similar digits are placed close together. The whole distribution is also shifted away from the origin (centred near $z_1 \approx 1.3$), so the KL term has not fully pulled it to $\mathcal{N}(0, I)$ in 15 epochs.

19. Generating New Images with the VAE

Sample points from

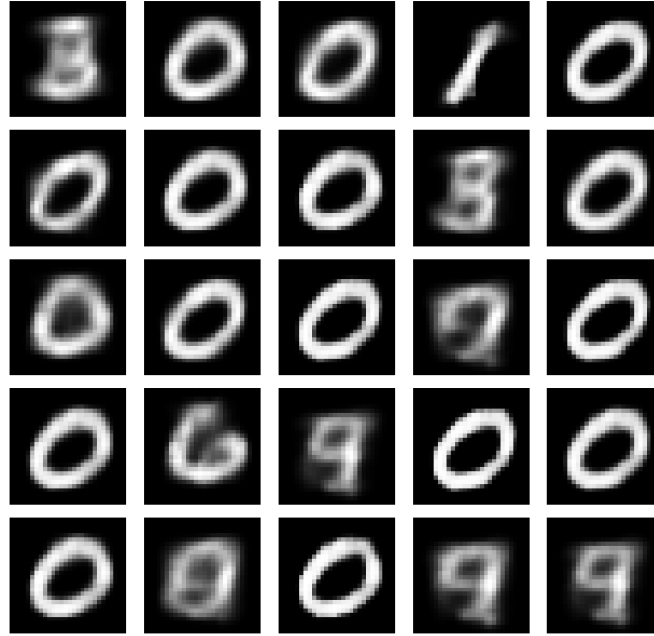
$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Plot 7: Randomly Generated VAE Images ($z \sim \mathcal{N}(0, I)$)



Inference: Most generated images are digit-like and recognisable (mainly 0, and some 1, 9, 6 and 3/8-like shapes), but they are blurry. Diversity is low since a large number of samples are 0s, because the learned latent cloud is not centred at the origin and $\mathcal{N}(0, I)$ samples fall mostly in the region of 0. A few samples (blends of 3/5/8) are ambiguous and unrealistic.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

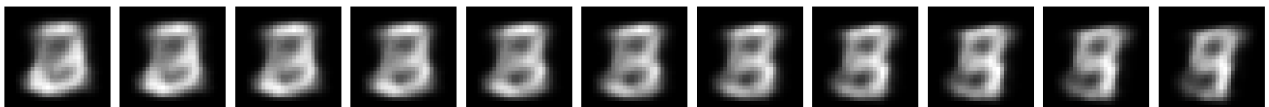
$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

Plot 8: Latent Space Interpolation (digit 3 $\rightarrow$ digit 8)



Inference: The decoded image changes gradually from the digit 3 to the other digit as α goes from 0 to 1, with no sudden jumps. The intermediate images are blurry but still look like digit-like shapes. This shows that the latent space is continuous, which is expected because of the KL regularisation.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.0258	0.0659	0.6979	211,040
Convolutional AE	0.0029	0.0158	0.9717	74,497
Denoising CAE	0.0052	0.0226	0.9402	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	160.59
KL Loss	5.68
Total Loss	166.27
Test MSE	0.0490
Test MAE	0.1147
Mean SSIM	0.4118

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

Plot: VAE Training and Validation Reconstruction Loss



Inference: The VAE reconstruction loss decreases from about 299 to 164 (training) and 162 (validation) over 15 epochs, and the validation curve stays close to the training curve, so there is no overfitting. The KL loss increases from about 2.4 to 5.7 as the encoder starts using the latent space, which is the usual trade-off between the two terms. The loss is still decreasing at epoch 15.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

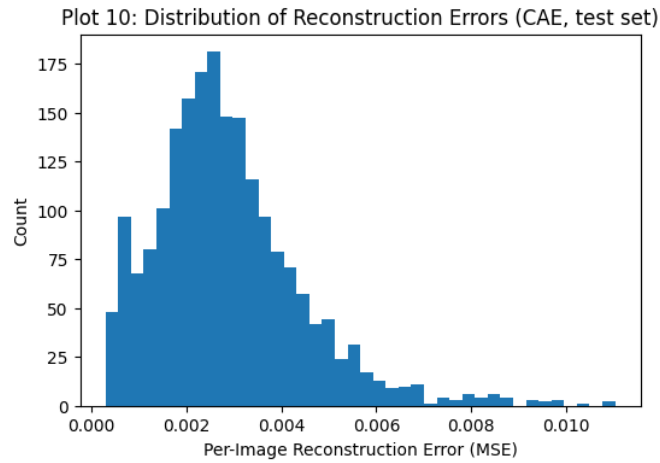
23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

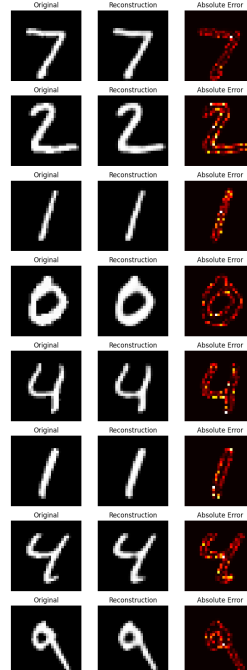
$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.



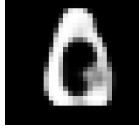
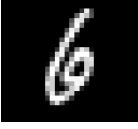
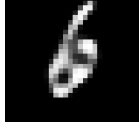
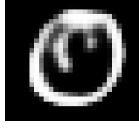
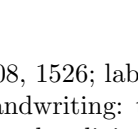
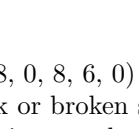
The per-image error (CAE, test set) has mean 0.0029, standard deviation 0.0016, minimum 0.0003 and maximum 0.0110. Most images lie between about 0.001 and 0.005, so the distribution is concentrated with a long right tail. The high-error samples (above about 0.008) are mostly unusual or messy handwriting, shown in the next section. The error maps below show the error is mostly along stroke edges.



Inference: Low-error images look almost identical to the input, while high-error images have visible blur or broken strokes. So a higher reconstruction error matches lower visual quality.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.
Display:

Sample	Original	Reconstruction
1		
		
2		
		
3		
		

The five images (test indices 543, 895, 1125, 1908, 1526; labels 8, 0, 8, 6, 0) have errors between 0.0098 and 0.0110. They are difficult because they have unusual handwriting: thick or broken strokes, extra loops and very slanted or stretched shapes. The CAE has mostly seen regular digits, so it smooths out these odd details and the result is blurry or slightly wrong.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
FC Autoencoder	0.0258	0.0659	0.6979	211,040	13.0 s
Conv. Autoencoder	0.0029	0.0158	0.9717	74,497	18.5 s
Denoising CAE	0.0052	0.0226	0.9402	74,497	15.7 s
VAE	0.0490	0.1147	0.4118	134,165	19.3 s

The values must be obtained from the student's own execution.

The CAE gives the best reconstruction. The denoising CAE is slightly worse because it is evaluated on noisy inputs ($\sigma = 0.2$). The VAE has the highest error because of the 2-D latent bottleneck and the KL constraint.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and

- differences between deterministic and probabilistic latent representations.

Inferences:

1. **Latent dimension:** MSE decreases and SSIM increases as the latent size grows (FC-AE: MSE 0.0519 to 0.0261, SSIM 0.377 to 0.684 from $d_z = 2$ to 32). A larger latent keeps more information, but with diminishing returns.
2. **FC vs convolutional:** the CAE has 9 times lower MSE and SSIM 0.97 vs 0.70 with fewer parameters, because convolutions keep spatial structure.
3. **Gaussian noise level:** MSE and MAE increase and SSIM falls (0.956 to 0.879) from $\sigma = 0.1$ to 0.3, since more noise hides more of the digit.
4. **Denoising ability:** the denoising CAE recovers the digit structure even at $\sigma = 0.3$, with some loss of fine detail.
5. **VAE latent distribution:** the encoder outputs a mean and variance per image and the KL term pushes it towards $\mathcal{N}(0, I)$ (KL ≈ 5.7 on test data).
6. **2-D latent space:** digits 0, 1 and 6 form fairly separate regions, while 4/7/9 and 2/3/5/8 overlap. The cloud is shifted from the origin.
7. **Generated samples:** they look like digits but are blurry and not very diverse in the 25-sample grid (many 0s).
8. **Interpolation:** the decoded images change smoothly with α , showing a continuous latent space.
9. **Error vs visual quality:** images with high per-image error (up to 0.011) are the ones with blur or broken strokes, so error matches visual quality.
10. **Deterministic vs probabilistic:** the AE gives a single point z and reconstructs well but cannot be sampled reliably, whereas the VAE gives a distribution which allows generation and interpolation, at the cost of blurrier reconstructions (SSIM 0.41).

27. Additional Latent-Dimension Study

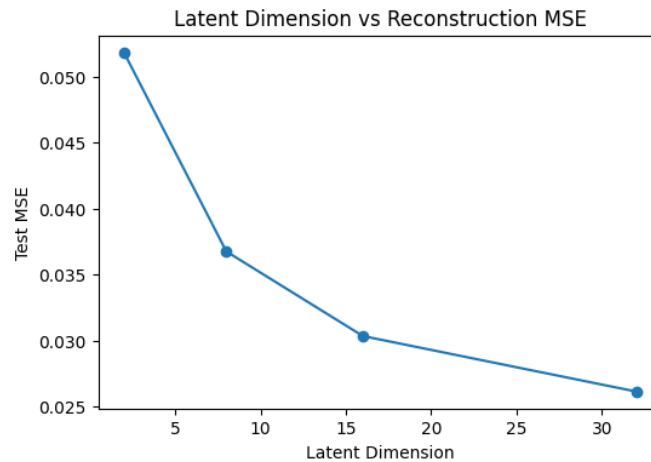
Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.0518	0.3774	210,130
8	0.0367	0.5801	210,520
16	0.0303	0.6428	211,040
32	0.0261	0.6836	212,080

Plot 10: Latent Dimension vs. Reconstruction MSE



Inference: MSE decreases steadily as the latent dimension increases (0.0518 for $d_z = 2$ to 0.0261 for $d_z = 32$), and SSIM increases from 0.377 to 0.684. A small latent gives high compression but loses details, while a bigger latent gives better reconstruction but less compression. The improvement slows down after 16, so the trade-off is diminishing returns. (Note: these runs used 8 epochs.)

28. Discussion Questions

Question 1. What is an autoencoder?

A neural network that is trained to reconstruct its own input. It compresses the input into a smaller latent representation (encoder) and rebuilds it from that (decoder).

Question 2. What is the purpose of the encoder?

It maps the high-dimensional input into a compact latent representation that keeps the important features.

Question 3. What is the purpose of the decoder?

It reconstructs the input from the latent representation as closely as possible.

Question 4. What is a latent representation?

A lower-dimensional vector (or a distribution in a VAE) that holds the essential information needed to reconstruct the input.

Question 5. Why is the latent representation usually smaller than the input?

So that the network cannot just copy the input and has to learn a compressed representation of the useful structure.

Question 6. What happens when the latent dimension is too small?

Too much information is lost, so the reconstruction error is high and the outputs are blurry. In my results $d_z = 2$ gave SSIM of only 0.377.

Question 7. What happens when the latent dimension is increased?

Reconstruction improves (lower MSE, higher SSIM), but the compression is less and the gains become smaller. In my results MSE went from 0.0518 to 0.0261 for $d_z = 2$ to 32.

Question 8. Why can a convolutional autoencoder preserve spatial information more effectively?

Convolution works on local neighbourhoods and shares weights across positions, so it captures strokes and edges. Flattening in the FC-AE destroys the 2-D structure.

Question 9. Differentiate an autoencoder and a denoising autoencoder.

A normal autoencoder reconstructs a clean input from the same clean input. A denoising autoencoder gets a corrupted input but is trained to output the clean image.

Question 10. Why is the clean image used as the target in denoising?

Because the model has to learn to remove the noise. The clean image is the noise-free output we want it to recover.

Question 11. What happens when the noise level is increased?

The task gets harder, MSE and MAE increase and SSIM decreases (SSIM 0.956 to 0.879 for $\sigma = 0.1$ to 0.3), and fine details are harder to recover.

Question 12. Why are MSE, MAE and SSIM useful for image reconstruction?

MSE and MAE measure pixel-wise error (MSE punishes large errors more). SSIM measures structural similarity, which pixel errors can miss, so together they give a better picture.

Question 13. What is a Variational Autoencoder?

A generative autoencoder that learns a probability distribution over the latent space instead of a single vector, so new images can be generated by sampling.

Question 14. How does a VAE differ from a conventional autoencoder?

A normal AE gives a fixed $z = f(x)$. A VAE outputs μ and $\log \sigma^2$, samples z from them, and adds a KL term to the loss.

Question 15. Why does a VAE learn a distribution instead of a single latent vector?

It makes the latent space continuous and regular, so points near each other decode to similar images and sampled points give valid outputs.

Question 16. What is the reparameterization trick?

Writing the sample as $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. The randomness is moved to ϵ , so gradients can flow through μ and σ in backpropagation.

Question 17. Why is ϵ sampled from a standard normal distribution?

So the random part does not depend on the trainable parameters, and z still has the required mean μ and variance σ^2 .

Question 18. What is the role of KL divergence in the VAE loss?

It keeps the learned distribution $q_\phi(z|x)$ close to the prior $\mathcal{N}(0, I)$, which keeps the latent space smooth and allows sampling.

Question 19. What is the prior distribution used in the VAE?

A standard normal distribution, $p(z) = \mathcal{N}(0, I)$.

Question 20. Why is a two-dimensional latent space useful for visualization?

It can be plotted directly as a scatter plot, so we can see how digits are grouped and whether they overlap.

Question 21. What does latent-space interpolation demonstrate?

That the latent space is smooth: moving between two latent points gives a gradual change between the two decoded images.

Question 22. Why can a VAE generate new images?

Because the latent space is regularised towards $\mathcal{N}(0, I)$, so a random sample from it can be passed through the decoder to get a new image.

Question 23. Why might some VAE-generated images be unclear?

Some samples fall in low-density regions or between digit clusters, so the decoder gives blurry or mixed shapes. The reconstruction loss also favours blurry averages.

Question 24. What is the difference between reconstruction and generation?

Reconstruction passes a given image through the encoder and decoder to reproduce it. Generation decodes a new sampled z that does not come from any specific image.

Question 25. Why should test images remain unseen during training?

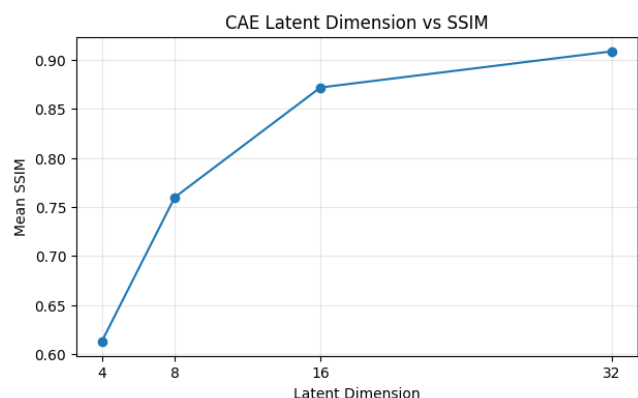
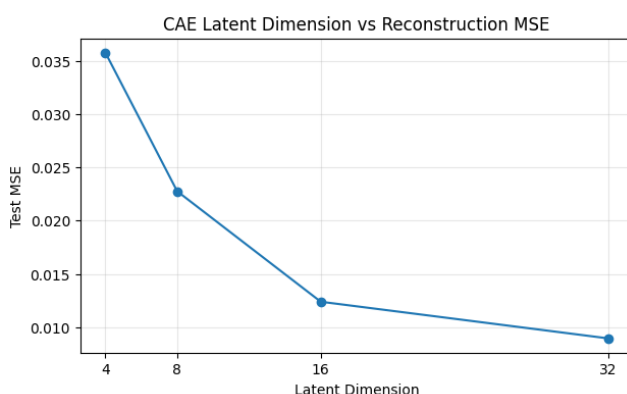
To get an unbiased estimate of performance on new data. If they were used in training, the MSE, MAE and SSIM would be too optimistic.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

Exercise 1: CAE with latent dimensions 4, 8, 16 and 32

Latent Dimension	MSE	SSIM	Parameters
4	0.0357	0.6128	102,725
8	0.0227	0.7598	127,817
16	0.0124	0.8719	178,001
32	0.0090	0.9088	278,369



Inference: MSE decreases (0.0357 to 0.0090) and SSIM increases (0.613 to 0.909) as the latent dimension increases from 4 to 32, but the parameter count almost triples. Small latents compress more but lose detail. The CAE with a 32-D bottleneck is also better than the FC-AE with the same latent size (SSIM 0.909 vs 0.684). (8 epochs used.)

Exercise 2: Gaussian vs salt-and-pepper noise

Noise Type	Level	MSE	MAE	SSIM
Gaussian	0.10	0.0041	0.0191	0.9561
Gaussian	0.20	0.0051	0.0226	0.9394
Gaussian	0.30	0.0077	0.0302	0.8799
Salt-and-Pepper	0.05	0.0047	0.0205	0.9472
Salt-and-Pepper	0.10	0.0059	0.0236	0.9272
Salt-and-Pepper	0.20	0.0101	0.0343	0.8381

Inference: For both types the error rises and SSIM falls as the level increases. At similar levels salt-and-pepper is slightly harder (SSIM 0.927 at $p = 0.10$ vs 0.939 at $\sigma = 0.2$), and at the highest level it is clearly worse (MSE 0.0101, SSIM 0.838) than Gaussian at $\sigma = 0.3$ (MSE 0.0077, SSIM 0.880), since salt-and-pepper replaces pixels fully with 0 or 1.

Exercise 3: Denoising AE trained at two noise levels (SSIM)

Training σ	Evaluation σ	SSIM
0.1	0.1	0.9532
0.1	0.2	0.8962
0.1	0.3	0.7107
0.3	0.1	0.9383
0.3	0.2	0.9281
0.3	0.3	0.9008

Inference: The model trained at $\sigma = 0.1$ is best on its own noise level (0.953) but drops sharply on $\sigma = 0.3$ (0.711). The model trained at $\sigma = 0.3$ is a bit lower at low noise (0.938 at $\sigma = 0.1$) but stays good across all levels (0.901 at $\sigma = 0.3$). So a model trained on stronger noise transfers better to other noise levels than one trained on weak noise.

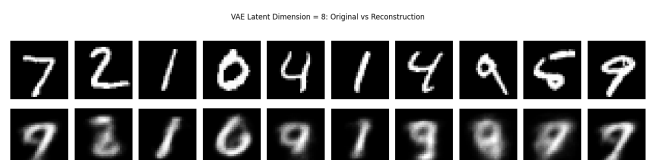
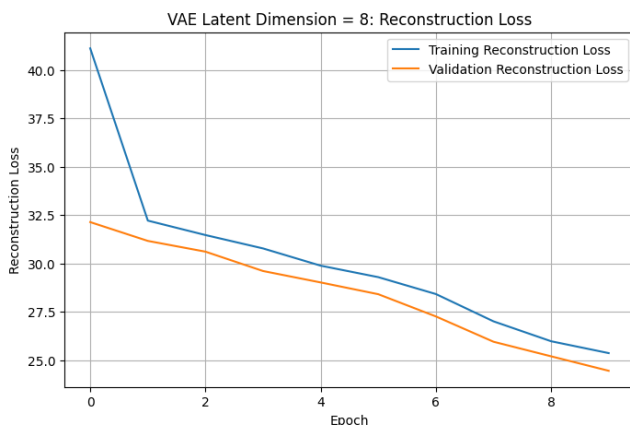
Exercise 4: UpSampling decoder vs transposed convolution decoder

Decoder	MSE	MAE	SSIM
UpSampling (main CAE)	0.0029	0.0158	0.9717
Conv2DTranspose	0.0035	0.0174	0.9658

Inference: Both decoders give very similar results. The UpSampling + Conv decoder is slightly better (MSE 0.0029 vs 0.0035), although the transposed-convolution model was trained for only 8 epochs compared to 15 for the main CAE, so the comparison is not perfectly equal. Transposed convolutions can cause checkerboard artifacts, but no strong artifacts were noticeable here.

Exercise 5: VAE with latent dimension 2 and 8

Model	MSE	MAE	SSIM
VAE, $d_z = 2$ (conv)	0.0490	0.1147	0.4118
VAE, $d_z = 8$ (dense)	0.0460	0.1068	0.4493



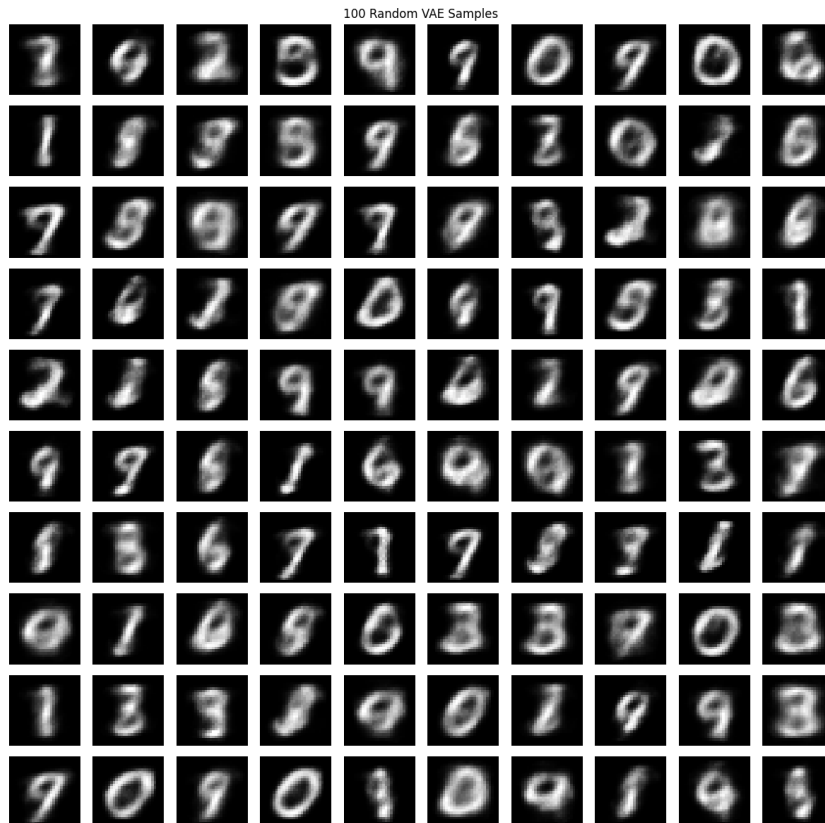
Inference: The 8-D VAE reconstructs slightly better (MSE 0.0460 vs 0.0490, SSIM 0.449 vs 0.412) as it has more capacity, but the 2-D VAE can be plotted directly and its class structure can be seen. The 8-D latent cannot be visualised. (The 8-D model used a dense architecture, so this is only an approximate comparison.)

Exercise 6: Effect of increasing the KL-loss contribution

β	Reconstruction Loss	KL Loss	Total Loss
0.1	not logged	1.741	8.751
1.0	not logged	1.833	8.856
2.0	not logged	1.506	8.702

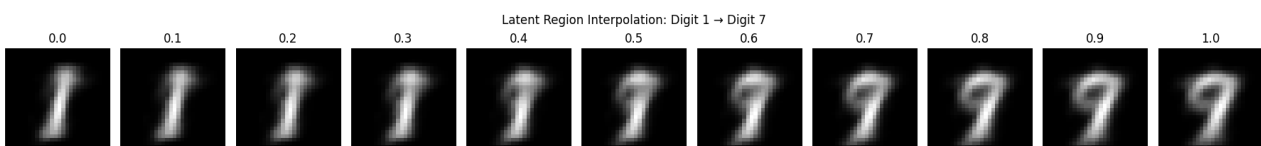
Inference: In my run the β value was set as a model attribute but the training loss did not use it (weight was effectively 1), so the three runs are basically repeats and the differences (KL 1.5–1.8) are just run-to-run variation, not an effect of β . The reconstruction loss was also not logged by `evaluate`. Theoretically, a larger β pushes the latent closer to $\mathcal{N}(0, I)$ (smaller KL, smoother latent space) but gives blurrier reconstructions, while a smaller β gives better reconstruction with a less regular latent space.

Exercise 7: Larger grid of VAE samples



Inference: The 100 samples (from the 8-D VAE decoder) show much more variety than the 25-sample grid and include most digits such as 0, 1, 2, 3, 5, 6, 7 and 9. Many are recognisable, but some are blurry or ambiguous (mixes of 3/5/8 and 4/9), and a few look similar to each other. A larger grid makes both the diversity and the unclear samples more visible.

Exercise 8: Interpolation between digit regions



Inference: The mean latent points of digit 1 and digit 7 (2-D VAE) were interpolated. The image changes smoothly from a straight 1 to a 7-like shape (a top bar slowly appears), and the middle images are blurry but still look like

digit-like strokes. Near $\alpha = 1$ the result looks like a blurry 7 (slightly 9-like), which is because the classes overlap in the 2-D latent space.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>

32. Source Code

https://github.com/ashrithaxx/Deep-Learning-Lab/tree/main/Experiment_7